

Tarea 1 — Parte Teórica

GitHub, Pytest y Flake8

Estudiantes: Isaac Cascante Oreamuno, Alejandro Pereira Gutiérrez

1. Explique la principal utilidad de git como herramienta de desarrollo de código.

Git es un sistema de control de versiones distribuido. Su principal utilidad es llevar un registro histórico y ordenado de todos los cambios que se hacen sobre el código de un proyecto a lo largo del tiempo. Esto permite consultar quién cambió qué y cuándo, comparar versiones y regresar a un estado anterior si algo se rompe. Además, al ser distribuido, cada desarrollador tiene una copia completa del historial, lo que facilita que varias personas trabajen en paralelo y luego integren su trabajo sin perder cambios ni sobrescribir el de los demás.

2. ¿Qué es un commit?

Un commit es una instantánea (snapshot) del estado del proyecto en un momento determinado. Cuando se realiza, git guarda de forma permanente los cambios preparados junto con un mensaje descriptivo, el autor, la fecha y un identificador único llamado hash. Es la unidad básica de la historia de un repositorio: cada commit representa un punto al que se puede volver.

3. ¿Qué es un branch?

Un branch (rama) es una línea de desarrollo independiente que parte de un punto del historial. Permite trabajar en una nueva funcionalidad o corrección de forma aislada, sin afectar la rama principal (master o main). Cuando el trabajo está listo y probado, sus cambios se integran de vuelta a la rama principal mediante una fusión (merge).

4. En el contexto de GitHub, ¿qué es un Pull Request?

Un Pull Request (PR) es una solicitud para integrar los cambios de una rama en otra, típicamente hacia master. Funciona como un mecanismo de revisión y colaboración: muestra las diferencias entre ambas ramas, permite comentar y discutir el código y ejecutar verificaciones automáticas antes de aprobar. Una vez aceptado, el PR se fusiona (merge) e incorpora los cambios.

5. Si un usuario quiere “Actualizar su repositorio contra el Branch master”, ¿qué debe hacer?

Debe traer los cambios más recientes del repositorio remoto e incorporarlos a su copia local. En la práctica se hace con `git pull origin master`, que combina dos operaciones: `git fetch` (descarga los cambios del remoto) y `git merge origin/master` (los fusiona con la rama local).

6. ¿Bajo qué condiciones Git necesita apoyo de un humano para integrar cambios locales con remotos?

Git necesita intervención humana cuando ocurre un conflicto de fusión (merge conflict). Esto sucede cuando dos cambios modifican la misma parte (las mismas líneas) de un mismo archivo de forma incompatible, o cuando una versión modifica un archivo que la otra eliminó. En esos casos git no puede decidir automáticamente cuál versión conservar, por lo que marca el conflicto y pide a la persona resolverlo manualmente y confirmarlo con un commit.

7. ¿Qué es una Prueba Unitaria o Unittest?

Es una prueba automatizada que verifica el correcto funcionamiento de la unidad más pequeña de código (normalmente una función o método) de forma aislada del resto del sistema. Consiste en ejecutar esa unidad con entradas conocidas y comprobar que produce la salida esperada. Sirve para detectar errores de forma temprana, documentar el comportamiento esperado y dar confianza al modificar el código.

8. Bajo el contexto de pytest, ¿cuál es la utilidad de un “assert”?

En pytest, **assert** es la instrucción con la que se comprueba que una condición sea verdadera. Si se cumple, la prueba continúa; si es falsa, pytest marca la prueba como fallida y muestra información detallada sobre los valores involucrados. Es el mecanismo central de verificación: confirma que el resultado real de una función coincide con el esperado, por ejemplo `assert resultado == valor_esperado`.

9. ¿Qué es Flake8?

Flake8 es una herramienta de análisis estático (linter) para Python que revisa el código sin ejecutarlo. Verifica que cumpla la guía de estilo PEP8 (espaciado, longitud de líneas, indentación) y detecta problemas como errores de sintaxis, variables o imports sin usar y funciones demasiado complejas. Internamente combina pycodestyle, pyflakes y mccabe.

10. Comente sobre la utilidad de la aleatorización en pruebas de código.

La aleatorización consiste en generar entradas aleatorias o variadas en cada ejecución de las pruebas, en lugar de usar siempre los mismos valores fijos. Así el código se prueba con muchos más casos de los que se escribirían manualmente, lo que aumenta la probabilidad de descubrir errores en situaciones límite no anticipadas y evita que la solución esté ajustada solo a unos pocos valores. Como buena práctica, cuando una prueba aleatoria falla conviene poder reproducir el caso (por ejemplo fijando una semilla) para depurar el error.